

NOMBRE: MARTINEZ HUICAB CHRISTOPHER

MATRICULA: 67374

NOMBRE DEL PROFESOR: **Sergio A Noh Puch**

NOMBRE DE LA MATERIA: **Concurrencia y Paralelismo**

FECHA DE ENTREGA: **17/10/2026**

NOMBRE DE LA INSTITUCIÓN: UNIVERSIDAD AUTÓNOMA DE CAMPECHE (CAMPUS V)

NOMBRE DE LA CARRERA: INGENIERÍA EN TECNOLOGÍA DE SOFTWARE

UAC

Universidad Autónoma de Campeche

"Del Enigma sin Albas a Triángulos de Luz"



Flujo utilizado	Tamaño (Elementos)	Ejec. (1 ms)	Ejec. (2 ms)	Ejec. (3 ms)	Promedio (ms)
FileWriter	1000000	1250	1180	1210	1213
BufferedWriter	1000000	45	38	42	41
FileReader	1000000	410	405	415	410
BufferedReader	1000000	22	19	25	22
FileOutputStream	1000000	850	820	840	836
BufferedOutputStream	1000000	12	10	15	12
FileInputStream	1000000	380	390	385	385
BufferedInputStream	1000000	15	12	14	13

Respuestas a las Preguntas de Análisis

1. ¿Qué diferencia observaste entre usar flujos con buffer y sus equivalentes directos? ¿Por qué ocurre?

Diferencia: Los flujos con buffer son drásticamente más rápidos (hasta 20-50 veces más rápidos en escritura/lectura) que los flujos directos.
Por qué ocurre: Los flujos directos (FileWriter, FileOutputStream) realizan una llamada al sistema operativo (syscall) por cada byte o carácter escrito/leído. Esto genera un cuello de botella gigante por el cambio de contexto entre la JVM y el Kernel. Los flujos con buffer (BufferedWriter, etc.) acumulan los datos en una memoria intermedia (RAM) y realizan una sola llamada al sistema operativo para escribir bloques grandes de datos (normalmente de 8 KB en 8 KB), reduciendo drásticamente las operaciones de E/S.

2. ¿Cómo varía el tiempo de ejecución al aumentar n? ¿La relación es lineal? ¿Se comporta igual en lectura que en escritura?

Variación: El tiempo aumenta a medida que n crece.
Relación: Es prácticamente lineal. Si duplicas n, el tiempo se duplica aproximadamente. Sin embargo, en los flujos con buffer, al principio hay un pequeño "costo fijo" (creación del buffer), pero luego el crecimiento es lineal.
Lectura vs Escritura: El comportamiento es similar, pero la lectura suele ser ligeramente más rápida que la escritura en discos magnéticos (aunque en SSD la diferencia es menor).

La lectura con FileInputStream directo suele ser algo más lenta que la escritura directa porque el sistema operativo debe esperar a que el disco físico entregue el byte.

3. ¿Existe diferencia de rendimiento entre trabajar con caracteres (texto) y con bytes (binario)? ¿A qué se debe?

Diferencia: El texto es generalmente más lento que el binario.
Por qué: FileWriter/FileReader realizan una conversión de codificación (encoding/decoding) de caracteres a bytes y viceversa (usando el charset por defecto, como UTF-8). Esto consume CPU. FileOutputStream/FileInputStream mueven bytes "crudos" sin procesarlos, por lo que son más eficientes computacionalmente.

4. ¿Qué tamaño tiene el buffer por defecto en Buffered...? ¿Puedes cambiarlo?

Tamaño por defecto: En Java, el tamaño por defecto es de 8192 bytes (8 KB) (o 8192 caracteres para BufferedWriter).
¿Se puede cambiar?: Sí. Todos los constructores de clases Buffered... tienen una sobrecarga que acepta un entero int para definir el tamaño del buffer (ej. new BufferedWriter(new FileWriter(ruta), 16384)).

5. ¿Por qué BufferedReader ofrece readLine() y BufferedInputStream no?

Razón: BufferedReader es un flujo de caracteres, por lo que entiende conceptos como "línea" (delimitada por \n o \r\n) y codificaciones. BufferedInputStream es un flujo de bytes puros; un byte no tiene un significado inherente de "salto de línea" hasta que se interpreta con un charset. Ofrecer readLine() en un flujo de bytes sería incorrecto porque no sabría cómo interpretar la codificación (UTF-8, UTF-16, ASCII, etc.).
Implicación: Para leer texto línea a línea de forma eficiente, debes usar BufferedReader. Si usas BufferedInputStream, tendrías que leer byte a byte, detectar manualmente los saltos de línea (por ejemplo, byte 10 o 13) y luego convertir esos bytes a String manualmente.

Conclusión Personal

En Java, el uso de buffers es una práctica obligatoria cuando se realizan operaciones de Entrada/Salida (I/O) que involucran archivos. La diferencia de rendimiento entre leer un byte a la vez desde el disco versus leerlo desde memoria RAM es abismal, como demuestran los benchmarks.
Para archivos de texto, BufferedReader y BufferedWriter son la elección estándar, especialmente porque BufferedReader nos facilita la vida con readLine(). Para datos binarios (imágenes, audio, serialización de objetos), BufferedInputStream y BufferedOutputStream son los reyes, ya que evitan el costo de codificación de caracteres y minimizan los accesos al disco.
La única desventaja del buffer es un ligero aumento en el uso de memoria (8 KB por flujo, algo despreciable hoy en día) y el riesgo de perder datos si el programa falla antes de que el buffer se vacíe (flush). Sin embargo, gracias al bloque try-with-resources, Java cierra y vacía el buffer automáticamente, haciendo que esta técnica sea segura y eficiente.